VEXON DEVELOPER HANDBOOK · GUIDE 02

JavaScript, TypeScript and React in VEXON

The frontend language and libraries, taught through the real web dashboard in client/

Written for	A student who can write basic code and wants to read and safely change VEXON's web dashboard
Project	VEXON / CampusConnect (repository: this folder's parent project)
Evidence	Generated from a direct inspection of the repository source code
Repository state	branch fix/csrf-legacy-cookie, commit af623fb, plus 126 uncommitted working-tree change(s)
Generated	16 October 2026, 21:11
Rebuild	python docs/developer-handbook/build_handbook.py

Accuracy rule: every statement about VEXON in this guide is based on files in the repository. Anything that could not be confirmed is labelled 'unverified', 'planned' or 'hypothetical'. No secrets, passwords or real student data are reproduced.

Contents

Before you start . . . 3

JavaScript fundamentals, using VEXON code . . . 3

- Variables: const and let . . . 3
- Functions and arrow functions . . . 4
- Objects, destructuring and spread . . . 4
- Arrays: map, filter, some, includes . . . 4
- Promises and async/await . . . 5
- Modules: import and export . . . 5
- Optional chaining and nullish coalescing . . . 6

TypeScript: what it is, and why VEXON does not need it now . . . 6

React fundamentals . . . 6

- Components and JSX . . . 6
- Props . . . 7
- State with useState . . . 7
- Effects with useEffect . . . 7
- Custom hooks . . . 8

Routing and protected pages . . . 8

Redux Toolkit and RTK Query . . . 9

- The store and a slice . . . 9
- RTK Query endpoints and cache tags . . . 10

Forms, validation and error handling . . . 10

Loading and error states . . . 11

Real-time updates in the UI . . . 11

Keeping logic testable: plain modules . . . 11

Tracing a button click to an API call . . . 12

How to locate a UI bug . . . 12

Making a small change safely . . . 13

Common beginner mistakes (quick reference) . . . 13

Practice exercises . . . 13

Source references . . . 14

Before you start

The web dashboard lives in `client/`. It is a **React 18** single-page app, built with **Vite**, that manages data with **Redux Toolkit Query (RTK Query)**. This guide teaches only the language features and libraries the dashboard actually uses. Every example is copied from a real file, so you can open the file and compare.

> **Evidence status**
> **TypeScript is not used in VEXON.** The audit found no `.ts` or `.tsx` source files in `client/`, `server/` or `mobile/`, and no `tsconfig.json`. Everything is plain JavaScript (`.js`, `.jsx`, `.mjs`). A short section below explains what TypeScript would add, so you can recognise it elsewhere. You do not need it to maintain VEXON.

How to read the examples: a file path, then the code, then "line by line", then the common mistakes, then an exercise with its expected result. Do the exercises in VS Code with the project open. None of them changes the app permanently unless the exercise says so, and those changes are easy to undo with `git checkout -- <file>`.

JavaScript fundamentals, using VEXON code

Variables: const and let

`const` names a value that will not be *re-assigned*. `let` is for values that change. VEXON almost always uses `const`. `var` is never used.

```js
// client/src/utils/constants.js
export const ROLES = ['student', 'club_admin', 'faculty', 'hod', 'principal', 'admin', 'security',
    'dean', 'ao', 'chairman', 'warden', 'alumni', 'coe', 'office', 'transport'];
export const STAFF = ['admin', 'faculty', 'hod'];
export const STAFF_VIEW = [...STAFF, 'principal'];
```

Line by line:

- `export` makes the name importable from other files (see *Modules* below).
- `const ROLES = [...]` is an array of 15 strings. `const` stops `ROLES = something else`, but the array *contents* could still be changed with `ROLES.push(...)`. VEXON never does that.
- `[...STAFF, 'principal']` uses the **spread** syntax: it copies the items of `STAFF` into a new array and adds one more.

> **Common mistake**
> People think `const` makes an object or array frozen. It does not. It only stops re-assignment. Also: this client list must stay in step with `ROLES` in `server/src/constants.js`. If you add a role on the server and forget the client, menus will hide things the server allows.

> **Exercise**
> Open `client/src/utils/constants.js` and find `TRANSPORT_VIEW`. Then open `server/src/constants.js` and find `TRANSPORT_VIEW_ROLES`. **Expected result:** both contain the same 12 roles. Alumni, COE and office are missing from both.

Functions and arrow functions

```
js
// client/src/utils/format.js
export const errMsg = (err, fallback = 'Something went wrong') =>
  err?.data?.message || err?.error || (err?.status === 'FETCH_ERROR' ? 'Cannot reach the server' :
    fallback);
```

Line by line:

- `(err, fallback = 'Something went wrong') => ...` is an **arrow function** with a **default parameter**. If the caller does not pass `fallback`, it gets that text.
- With no `{ }` after `=>`, the expression's value is returned automatically.
- `err?.data?.message` uses **optional chaining**: if `err` or `err.data` is `null` or `undefined`, the result is `undefined` instead of a crash.
- `a || b || c` returns the first "truthy" value, so the user sees the server's message first, then RTK's error text, then a network message, then the fallback.
- `cond ? x : y` is the **ternary** operator, a one-line if/else.

Why it matters: every failing request in the web app shows its toast through `errMsg`. If a user reports "Something went wrong", it means the server sent no `message`. Look at the Network tab (Guide 06).

Objects, destructuring and spread

```
js
// client/src/components/ui/form.jsx
export const Input = forwardRef(function Input({ label, error, hint, className, icon: Icon, ...props },
    ref) {
  return (
    <Field label={label} error={error} hint={hint} className={className}>
      ...
        <input ref={ref} className={cn('input', Icon && 'pl-11', error && 'input-error')} {...props} />
```

- `{ label, error, hint, ... }` in the parameter list is **object destructuring**. It pulls named properties out of the object the caller passed.
- `icon: Icon` renames the property `icon` to a local variable `Icon`. React needs component names to start with a capital letter.
- `...props` collects *all remaining* properties (such as `type`, `placeholder` and `onChange`), and `{...props}` spreads them onto the real `<input>`.
- `Icon && 'pl-11'` evaluates to `'pl-11'` when there is an icon and to `undefined` otherwise. The `cn()` helper drops falsy values.

On the server, the same idea protects the database. `pick(req.body, ['anonymous', 'category', ...])` in `server/src/utils/http.js` copies **only whitelisted keys**, so a user cannot sneak in `role: 'admin'` (mass-assignment protection).

Arrays: map, filter, some, includes

```
js
// client/src/App.jsx  (CoeConfine)
const rule = CONFINED[role];
if (rule && !rule.pages.some((p) => pathname === p || pathname.startsWith(`${p}/`))) return <Navigate
    to={rule.home} replace />;
```

- `CONFINED[role]` reads an object property with a variable key. It is `undefined` for roles that are not confined.

- `.some(fn)` returns `true` if *at least one* item passes the test.
- `` ` ${p}/ is a **template literal**: back-ticks with ${} `` placeholders.

Other array methods you will see constantly: `.map()` (turn each item into something else, used to render lists in JSX), `.filter()` (keep only matching items), `.includes(x)` (membership test, used in `RequireAuth`: `roles.includes(user.role)`), and `.find()` (the first match).

> **Exercise**
> In `client/src/pages/Notifications.jsx`, find `data.items.map((n) => (`. What does each `n` represent, and which property becomes React's `key`? **Expected result:** each `n` is one notification document from `GET /api/notifications`, and `key={n._id}` (its MongoDB id) is the key.

Promises and async/await

Network calls take time, so they return a **Promise**, an object that will later hold either a result or an error. `await` pauses an `async` function until the promise settles. `try { } catch { }` catches the error case.

```
js
// client/src/pages/auth/Login.jsx  (inside onSubmit)
try {
  const data = await login(values).unwrap();
  sessionStorage.removeItem('campus_lockout');
  setRemainingAttempts(null);
  dispatch(setCredentials(data));
  toast.success(`Welcome back, ${data.user.name.split(' ')[0]}!`);
  navigate(location.state?.from || '/', { replace: true });
} catch (err) {
  const data = err?.data;
  if (err?.status === 423 || data?.locked) { ... }
```

- `login(values)` is an RTK Query mutation trigger. `.unwrap()` turns it into a normal promise that **throws** on an HTTP error, so `catch` runs.
- `dispatch(setCredentials(data))` stores the user and the access token in Redux.
- `navigate(location.state?.from || '/')` sends the user back to the page they originally wanted. `RequireAuth` saved it in `state.from`.
- Status `423` means the account is locked (see the server's `authController.login`).

A more advanced promise pattern: **share one in-flight request**. In `client/src/services/api.js`, `refreshSession()` stores the refresh promise in `refreshPromise`. Ten requests that all hit `401` at once therefore trigger **one** refresh call, not ten.

> **Common mistake**
> Forgetting `await`. You then get a Promise object instead of data, and errors escape your `try/catch`. Another mistake is forgetting `.unwrap()` on an RTK mutation: without it the promise never throws, and your `catch` block never runs.

Modules: import and export

```
js
// client/src/app/store.js
import { configureStore } from '@reduxjs/toolkit';
import authReducer from '../features/authSlice';
import { api } from '../services/api';
export const store = configureStore({ ... });
```

- `import { x } from '...'` imports a **named** export. The name must match exactly.
- `import authReducer from '...'` imports the **default** export. You choose the local name (`authSlice.js` ends with `export default authSlice.reducer`).
- A path starting with `.` or `..` is a file in the project. A bare name like `'@reduxjs/toolkit'` is a package in `node_modules`.
- The client and server both use ES modules: `"type": "module"` in their `package.json`. Server imports must include the `.js` extension (`'../utils/http.js'`). Vite lets the client omit it.

Optional chaining and nullish coalescing

`a?.b` is safe when `a` might be missing. `a ?? b` returns `b` only when `a` is `null` or `undefined`, unlike `||`, which also replaces `0`, `''` and `false`. In `client/src/features/authSlice.js`: `state.idleTimeoutSec = payload.idleTimeoutSec ?? null;`.

TypeScript: what it is, and why VEXON does not need it now

TypeScript is JavaScript plus **type annotations** that a compiler checks before the code runs. For example: `function errMsg(err: unknown, fallback: string = '...'): string`. It catches mistakes like passing a number where a string is expected.

VEXON gets some of that safety in other ways:

- **Mongoose schemas** (`server/src/models/*.js`) enforce field types and enums on the server.
- **express-validator** rules in `server/src/routes/index.js` check every request body.
- **Kotlin and Swift** in the mobile apps are statically typed.

> **Note**
> If you later add TypeScript, convert one leaf file at a time (`utils/format.js` to `format.ts`), add `typescript` and a `tsconfig.json` to `client/`, and keep `npm run build` green after each step. This is a **suggestion, not current project state**.

React fundamentals

Components and JSX

A React **component** is a function that returns **JSX**, an HTML-like syntax that compiles to JavaScript.

```jsx
// client/src/App.jsx
function NotFound() {
  return (
    <div className="card mx-auto max-w-md text-center">
      <p className="text-gradient text-6xl font-extrabold">404</p>
      <p className="mt-2 font-bold">Page not found</p>
      <p className="mt-1 text-sm muted">The page you're looking for doesn't exist.</p>
    </div>
  );
}
```

- JSX uses `className`, not `class`. The class names are **Tailwind CSS** utilities (`mx-auto`, `text-sm`) plus custom classes defined in `client/src/index.css` (`card`, `muted`).

- A component must return **one** root element.

Props

Props are a component's inputs, passed like HTML attributes. `Field` in `client/src/components/ui/form.jsx` receives `{ label, error, hint, children, className }`. `children` is whatever you put between the opening and closing tags.

```jsx
export function Field({ label, error, hint, children, className }) {
  return (
    <div className={className}>
      {label && <label className="label">{label}</label>}
      {children}
      {error ? (<p className="mt-1.5 text-xs font-medium text-rose-500">{error.message || error}</p>)
             : hint ? (<p className="mt-1.5 text-xs muted">{hint}</p>) : null}
    </div>
  );
}
```

`{label && <label>...}` is **conditional rendering**: the label is drawn only if one was given.

State with useState

```jsx
// client/src/pages/Notifications.jsx
const [filter, setFilter] = useState('all');
const [page, setPage] = useState(1);
const { data, isLoading } = useGetNotificationsQuery({ unread: filter === 'unread' ? 'true' : undefined,
    page, limit: 20 });
```

- `useState('all')` returns the current value and a setter. Calling `setFilter('unread')` re-renders the component with the new value.
- Because `filter` and `page` are passed to the query hook, changing them **automatically triggers a new API call** with the new parameters.
- In the tab `onChange` handler, the page resets to 1 (`setPage(1)`) when the filter changes. That avoids showing "page 3" of a shorter list.

> **Common mistake**
> Changing state directly (`page = 2` or `data.items.push(x)`). React does not notice, so nothing re-renders. Always call the setter, or, for server data, trigger a mutation and let RTK Query refetch.

Effects with useEffect

`useEffect` runs code *after* rendering, for work like starting timers, subscribing to events, or loading the session.

```jsx
// client/src/App.jsx
useEffect(() => {
  refreshSession().then((session) => {
    if (session?.accessToken) dispatch(setCredentials(session));
    dispatch(sessionChecked());
  });
}, [dispatch]);
```

- This runs once when the app starts. It asks `/api/auth/refresh` whether the httpOnly cookie still holds a valid session, which is how a page reload keeps you signed in.
- Until `sessionChecked()` sets `ready = true`, `App` shows a spinner instead of routes.
- `[dispatch]` is the **dependency array**: the effect re-runs only when one of these values changes.

Cleanup example from `Login.jsx`. The lockout countdown starts an interval and **returns a cleanup function**, so the timer stops when the component disappears:

```jsx
timerRef.current = setInterval(() => { ... }, 1000);
return () => clearInterval(timerRef.current);
```

> **Common mistake**
> Missing cleanup. The interval keeps running after you leave the page, leaking memory and calling `setState` on an unmounted component. Every `setInterval`, socket listener or `watchPosition` in an effect needs a matching cleanup.

Custom hooks

A **custom hook** is a function starting with `use` that bundles hook logic. VEXON's socket hooks in `client/src/services/socket.js` are good examples:

```js
export function useSocketEvent(event, handler) {
  const ref = useRef(handler);
  ref.current = handler;
  useEffect(() => {
    const fn = (...args) => ref.current?.(...args);
    if (!listeners.has(event)) listeners.set(event, new Set());
    listeners.get(event).add(fn);
    return () => listeners.get(event)?.delete(fn);
  }, [event]);
}
```

- `useRef` holds the latest `handler` without re-subscribing on every render.
- The effect registers a listener in a module-level `Map`, so it works even if the socket connects later.
- The cleanup removes the listener when the component unmounts. This is exactly the pattern the previous Mistake box describes.

Routing and protected pages

`client/src/main.jsx` wraps `<App />` in `<BrowserRouter>`. `client/src/App.jsx` declares the URL table with React Router v6:

```jsx
<Route path="/login" element={<GuestOnly><Login /></GuestOnly>} />
<Route element={<RequireAuth><CoeConfine><AppLayout /></CoeConfine></RequireAuth>}>
  <Route index element={<Dashboard />} />
  <Route path="admin/users" element={<RequireAuth roles={['admin']}><AdminUsers /></RequireAuth>} />
  <Route path="transport" element={<RequireAuth roles={TRANSPORT_VIEW}><Transport /></RequireAuth>} />
  ...
```

```jsx
```

```
function RequireAuth({ children, roles }) {
  const user = useSelector((s) => s.auth.user);
  const location = useLocation();
  if (!user) return <Navigate to="/login" replace state={{ from: location.pathname + location.search }}
    />;
  if (roles && !roles.includes(user.role)) return <Navigate to="/" replace />;
  return children;
}
```

- Not signed in: redirect to `/login`, remembering where you wanted to go.
- Wrong role: redirect to the dashboard.
- **Pages are lazy-loaded** (`const Events = lazy(() => import('./pages/events/Events'))`) inside `<Suspense fallback={<PageLoader />}>`, so each page's code downloads only when visited.
- `AppLayout` (in `client/src/components/layout/AppLayout.jsx`) is the shell with the sidebar and top bar. Child pages render inside its `<Outlet />`.

> **Key idea**
> Route guards are **user experience, not security**. A student can still type an admin URL into the address bar or call the API from the console. The real protection is `authorize(...)` and the scope checks on the server.

Redux Toolkit and RTK Query

The store and a slice

`client/src/app/store.js` creates one Redux store with three parts: `auth`, `ui` and `api` (the RTK Query cache).

```
js
// client/src/features/authSlice.js
const authSlice = createSlice({
  name: 'auth',
  initialState,
  reducers: {
    setCredentials(state, { payload }) {
      state.user = payload.user;
      state.accessToken = payload.accessToken;
      state.idleTimeoutSec = payload.idleTimeoutSec ?? null;
    },
    loggedOut(state) { state.user = null; state.accessToken = null; state.idleTimeoutSec = null; },
    ...
```

- `createSlice` generates **action creators** (`setCredentials`, `loggedOut`) and a reducer.
- Inside reducers you *may* write `state.user = ...`. Redux Toolkit uses the Immer library to turn that into an immutable update. This is the **only** place where "mutating" state is allowed.
- Components read state with `useSelector((s) => s.auth.user)` and change it with `dispatch(loggedOut())`.

RTK Query endpoints and cache tags

```js
// client/src/services/api.js
getNotifications: b.query({
  query: (params) => ({ url: '/notifications', params }),
  providesTags: ['Notification'],
}),
markNotificationRead: b.mutation({
  query: (id) => ({ url: `/notifications/${id}/read`, method: 'PATCH' }),
  invalidatesTags: ['Notification', 'Dashboard'],
}),
```

- A `query` reads data. RTK Query generates `useGetNotificationsQuery(params)`, which returns `{ data, isLoading, isFetching, error, refetch }` and caches the result.
- A `mutation` changes data. It generates `useMarkNotificationReadMutation()`, which returns `[trigger, { isLoading }]`.
- **Tags connect the two.** When `markNotificationRead` succeeds, every cached query that `providesTags: ['Notification']` (or `'Dashboard'`) is re-fetched automatically. You never refresh lists by hand.
- At the bottom of `api.js`, `export const { useLoginMutation, ... } = api` exports all the hooks.

> **Common mistake**
> Adding a new mutation but forgetting `invalidatesTags`. The save works on the server, but the list on screen stays stale until a reload. It looks like "the save didn't work". Check the tags first.

Forms, validation and error handling

VEXON uses **React Hook Form** (`useForm`). From `client/src/pages/auth/Login.jsx`:

```jsx
const { register, handleSubmit, formState: { errors } } = useForm({ defaultValues: { email: '', password: '' } });
...
<Input error={errors.email} {...register('email', {
  required: 'Email is required',
  pattern: { value: /^\S+@\S+\.\S+$/, message: 'Enter a valid email' },
})} />
```

- `register('email', rules)` connects the input to the form and declares client-side rules.
- `handleSubmit(onSubmit)` runs the rules first and calls `onSubmit(values)` only when they pass.
- `errors.email` holds the failed rule's message, which `Field` displays in red.

Client validation is only for **convenience**. The server repeats every rule in `server/src/routes/index.js` with express-validator. Login, for example, is checked with `body('email').trim().isEmail()`. If the server rejects a request, it answers `422` with `{ message, errors: [{ field, message }] }`, and `errMsg(err)` shows the message.

`ComplaintFormModal.jsx` shows a richer form: `watch('category')` makes the sub-category dropdown depend on the category, and a `useEffect` resets dependent fields when the category changes.

Loading and error states

The standard VEXON page pattern, from `Notifications.jsx`:

```jsx
{isLoading ? (
  <PageLoader />
) : data?.items?.length ? (
  <Card> ...list... </Card>
) : (
  <EmptyState ... />
)}
```

There are three states: **loading** (show a spinner), **has data** (show it) and **empty** (show a friendly message). Mutations show `loading` on their button (`<Button loading={marking}>`) and report failures with `toast.error(errMsg(e))`.

> **Tip**
> `isLoading` is true only for the *first* load. `isFetching` is also true during background refetches. Use `isFetching` if you want a small "refreshing" indicator.

Real-time updates in the UI

`AppLayout.jsx` connects the socket once per signed-in session and maps server events to cache tags:

```js
const LIVE_TAGS = {
  'announcement:changed': ['Announcement', 'Dashboard'],
  'gatepass:updated': ['GatePass'],
  'sos:new': ['SOSAlert'],
  ...
};
```

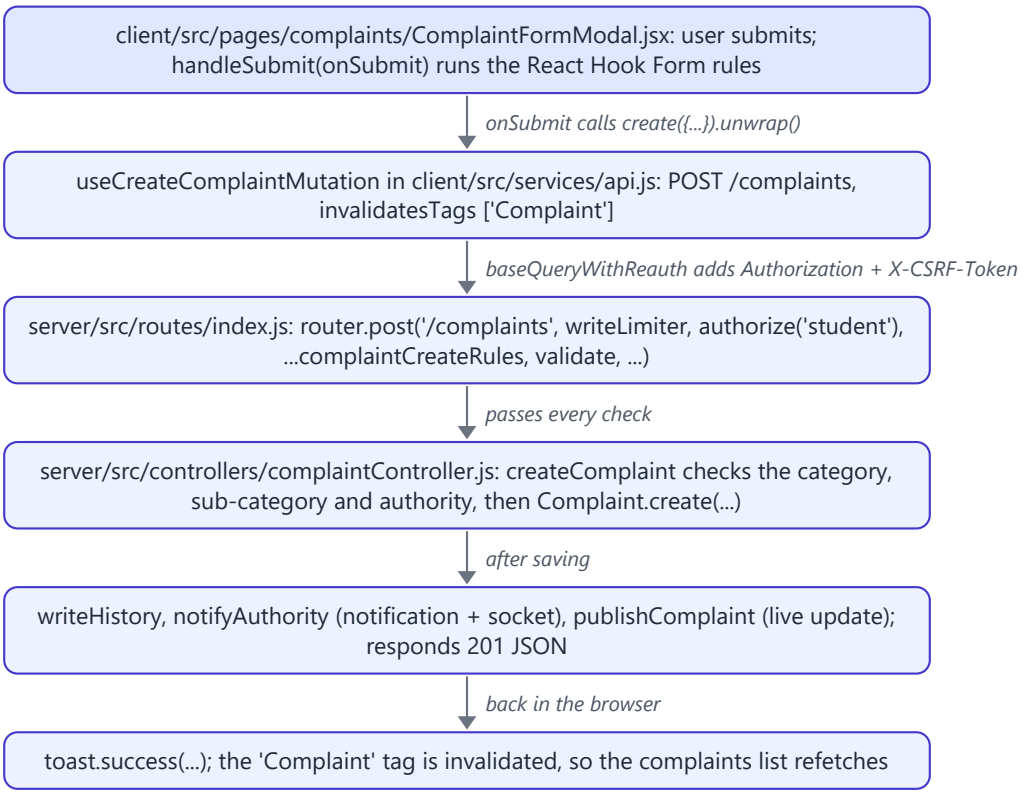
When the server emits `gatepass:updated`, the layout calls `dispatch(api.util.invalidateTags(['GatePass']))`, and every gate-pass list on screen re-fetches over REST. When the socket reconnects after a network drop, all live tags are refreshed so nothing is missed. Live bus positions are the exception: `client/src/pages/transport/Transport.jsx` subscribes with `useSocketEvent('bus:location', ...)` and moves the markers directly, without a REST round-trip.

Keeping logic testable: plain modules

Some logic is deliberately kept **outside React**, so it can be unit-tested with Node. `client/src/pages/transport/myLocation.js` says so in its header comment: "Free of React and of `window` globals (passed in) so it runs under node --test." Its test, `client/tests/myLocation.test.mjs`, passes a **fake** `navigator.geolocation`, so no real GPS is needed. Run `npm test --prefix client` and you will see those tests pass (46 tests passed during this audit).

Tracing a button click to an API call

Follow the "Register grievance" button from the screen to the database:



The general method works for **any** button:

1. Find the button text in the UI, then search for it in `client/src` (VS Code: Ctrl+Shift+F).
2. Find its `onClick` or `onSubmit` handler in that component.
3. Find the hook it calls (`useSomethingMutation`) and search for the endpoint name in `client/src/services/api.js` to get the URL and method.
4. Search for that URL in `server/src/routes/index.js` to see the middleware chain and the controller function.
5. Open the controller function and read it top to bottom.

How to locate a UI bug

1. **Reproduce it** and write down the exact steps, role and page.
2. Open the browser **DevTools Console**. A red error with a file and line from `client/src/...` points straight to the component (in development builds).
3. Open the **Network** tab. Did the request fail (4xx/5xx)? Then it is probably a server or permission issue (Guide 06). Did it succeed but the screen looks wrong? Then it is a UI or state issue.
4. Install the **React Developer Tools** and **Redux DevTools** browser extensions. Inspect the component's props, and see whether the RTK Query cache holds the data you expect.
5. Search the visible text in `client/src` to find the component.
6. Add a temporary `console.log` near the suspected line, reload, and read the value. **Remove it afterwards**: production builds strip console calls (`client/vite.config.js`), but the source should stay clean.

Making a small change safely

1. `git status`: start from a clean or understood working tree (see Guide 07).
2. Create a branch: `git switch -c fix/notification-subtitle`.
3. Make the smallest possible edit.
4. Run the app (`npm run dev`) and check the change **as the right role**.
5. Run `npm test --prefix client` and `npm run build --prefix client`.
6. Read `git diff` line by line before committing.

Exercise

Safe practice change. In `client/src/pages/Notifications.jsx`, change the `subtitle` text of `PageHeader`. Run `npm run dev`, sign in, and open Notifications. **Expected result:** the new subtitle appears and nothing else changes. Then undo it with `git checkout -- client/src/pages/Notifications.jsx` and confirm `git diff` shows nothing for that file.

Common beginner mistakes (quick reference)

Mistake	Symptom	Fix
Forgot `await` / `.unwrap()`	`catch` never runs; `[object Promise]` shown	Add `await` and `.unwrap()`
Mutated state directly	Screen does not update	Use the setter / dispatch / a mutation
Missing `key` in `.map()`	Console warning; list items behave oddly	`key={item._id}`
Effect without cleanup	Timers or listeners pile up; memory grows	Return a cleanup function
Wrong dependency array	Effect runs forever, or never re-runs	List every value the effect reads
Mutation without `invalidatesTags`	Saved, but the list is stale	Add the right tag
Trusting the client guard	"Security" that a console call bypasses	Enforce on the server
Role list not mirrored	Menu hidden but API allowed, or the reverse	Keep `client/src/utils/constants.js` in step with `server/src/constants.js`

Practice exercises

Exercise

1. Find an endpoint. Which hook does the Notifications page use to delete a notification, and which HTTP method and URL does it call? **Expected:** `useDeleteNotificationMutation`, `DELETE /api/notifications/:id`.

Exercise

2. Read a guard. Which roles can open `/transport/manage`? Find the answer in `App.jsx` and `constants.js`. **Expected:** `TRANSPORT_MANAGERS` = admin and transport.

Exercise

3. Trace the session restore. After a page reload, which function decides whether you are still signed in, and which URL does it call? **Expected:** `refreshSession()` in `client/src/services/api.js` calls `POST /api/auth/refresh`, triggered by the `useEffect` in `App.jsx`.

Exercise

4. Socket to cache. Which cache tags refresh when the server emits `holiday:changed`? **Expected:** `Holiday`, `Attendance`, `Timetable` (from `LIVE_TAGS` in `AppLayout.jsx`).

Exercise

5. Run the unit tests. Run `npm test --prefix client`. **Expected:** all tests pass (46 passed during this audit). Open `client/tests/busGeo.test.mjs` and explain one test in your own words.

Source references

`client/package.json` · `client/vite.config.js` · `client/src/main.jsx` · `client/src/App.jsx` · `client/src/app/store.js` · `client/src/features/authSlice.js` · `client/src/services/api.js` · `client/src/services/socket.js` · `client/src/components/layout/AppLayout.jsx` · `client/src/components/ui/form.jsx` · `client/src/utils/format.js` · `client/src/utils/constants.js` · `client/src/pages/Notifications.jsx` · `client/src/pages/auth/Login.jsx` · `client/src/pages/complaints/ComplaintFormModal.jsx` · `client/src/pages/transport/myLocation.js` · `client/tests/myLocation.test.mjs` · `server/src/routes/index.js` · `server/src/controllers/complaintController.js` · `server/src/utils/http.js`